EE627 — Class Summary

April 24 & May 1, 2026

Prof. Rensheng Wang

Contents

1 Course Context: Why We Keep All Submissions **2**
1.1 The Ensemble Intuition . . . 2
1.2 Implications for the Final Project . . . 2

2 Feature Engineering and Data Transformation **3**
2.1 Motivation . . . 3
2.2 Centering and Scaling . . . 3
2.3 Other Common Transformations . . . 3
2.4 Transformations to Resolve Outliers . . . 3

3 Feature Selection and Redundancy **5**
3.1 Noise Features . . . 5
3.2 Redundant Features . . . 5
3.3 Feature Selection Strategies . . . 5

4 Principal Component Analysis (PCA) **6**
4.1 High-Level Goal . . . 6
4.2 Formal Definition . . . 6
4.3 Eigenvalue Decomposition . . . 6
4.4 Geometric Interpretation . . . 6
4.5 Must Center and Scale Before PCA . . . 7
4.6 Caveats: When PCA Can Mislead . . . 7
4.7 How Many Components to Keep: The Scree Plot . . . 7
4.8 Worked Example: Stock Market (IBM, HP, Intel, JPM, BOA) . . . 7
4.9 Worked Example: Eigenfaces . . . 8
4.10 Practical Takeaway for the Project . . . 8

5 Supervised Learning with PySpark and MLlib (Homework 9) **9**
5.1 The Shift from Unsupervised to Supervised . . . 9
5.2 Why PySpark Classifiers? . . . 9
5.3 Worked Example: Bank Marketing Dataset . . . 9
5.4 Full Data Preparation Pipeline . . . 9
5.5 Four Classifiers: Code and Results . . . 10
5.6 Hyperparameter Tuning with Cross-Validation . . . 12
5.7 Applying to the Kaggle Project . . . 12

6 Ensemble Preview **14**

7 Key Reference **15**

1 Course Context: Why We Keep All Submissions

Both sessions opened with the same motivating theme: students working on the Kaggle recommendation project should keep every Kaggle submission they have ever made, regardless of how low the score is. The reason is the upcoming **ensemble** lecture, where all historical submissions will be fused into a single prediction that beats any individual one.

1.1 The Ensemble Intuition

Suppose you have two submissions: one scoring 0.90 and one scoring 0.80. Combining them yields a new prediction that can score 0.92 — higher than either alone. If you discard the weaker result, you lose the diversity that makes the improvement possible. The analogy from lecture: at the end of the semester every student stands "on stage" and the professor cares about *how many* predictions you have, not just which one is best.

1.2 Implications for the Final Project

- The final project deadline is **May 13**.
- Students are encouraged to keep trying new approaches even when results are worse than what they already have.
- Each new approach adds a diversified "vote" to the eventual ensemble, and more diversified votes raise the ceiling for the blended score.
- One-on-one sessions with the professor are available by email for individual project questions.

2 Feature Engineering and Data Transformation

2.1 Motivation

In the Kaggle project, every user–track pair is associated with several scores: an album (urban) score, an artist score, and various statistical summaries of the general (genre) scores (max, min, mean, variance, etc.). The heuristic approaches used in Homework 4 and the midterm combined these features by hand with manually chosen weights — e.g., $0.5 \times \text{urban} + 0.3 \times \text{artist} + 0.2 \times \text{general}$. The weights were reasonable but arbitrary. The goal of feature engineering is to either find better representations or generate more candidate features for the ensemble.

2.2 Centering and Scaling

The most fundamental transformation is **center-scaling** (standardization):

- **Center:** subtract the mean of each predictor so it has zero mean.
- **Scale:** divide by the standard deviation so each predictor has unit variance.

These steps make features comparable in magnitude and are required before PCA (see Section 4). The downside is a loss of interpretability, since values are no longer in the original units.

2.3 Other Common Transformations

Normalization to $[0, 1\ .]Whenafeaturespansahugerange(e.g., 0.1to1\,000\,000), subtracttheminimumanddividel$ $maxrange : rescaleseverythingto[0,\ 1]$.

Log scaling. When a count column is heavily right-skewed (e.g., one item has 1 000 000 ratings while most have 1–2), apply $\log(\cdot)$ to compress the range while preserving order. Monotonicity is maintained: $a > b \Rightarrow \log a > \log b$.

Derived features. Compute ratios or differences between existing columns (e.g., today's score divided by yesterday's). These expand the feature space without collecting new data.

Combination / synthesis. Form weighted averages of existing features to create new composite scores. This is the conceptual precursor to PCA.

2.4 Transformations to Resolve Outliers

The slides define an **outlier** as a sample that is exceptionally far from the mainstream of the data. Before acting on suspected outliers:

1. Verify the value is scientifically valid (e.g., a negative blood-pressure reading is likely a recording error).
2. Be cautious with small samples — apparent outliers may reflect a skewed distribution with too few observations to reveal the skewness.
3. Check whether the outlying points form a coherent cluster representing a genuine subpopulation (e.g., highly profitable customers who behave differently from the mainstream).

Some model families are naturally outlier-resistant:

- **Tree-based models** (decision trees, random forests, GBT) split on threshold comparisons; a single extreme value usually does not shift the split threshold enough to change the prediction.

- **Support vector machines** explicitly exclude a fraction of training samples (the support vectors are far from the decision boundary).

For models that *are* sensitive to outliers, the **spatial sign transformation** can be applied. Each sample $\mathbf{x}_i$ (a row of the predictor matrix) is projected onto a unit sphere:

$$x_{ij}^* = \frac{x_{ij}}{\sum_{j=1}^{p} x_{ij}^2}$$

This makes every sample the same distance from the center, neutralizing extreme rows. Important caveats:

- The spatial sign acts on the *entire row* as a group; it cannot be applied feature-by-feature.
- Features must be centered and scaled *before* applying the spatial sign, because the denominator measures distance to the center of the predictor's distribution.
- Removing individual features after the spatial sign transformation is problematic, since the transform entangles all predictors.

3 Feature Selection and Redundancy

3.1 Noise Features

A "noise feature" carries no information relevant to the target. The extreme example: if you are predicting Ivy League admissions, a column recording each applicant's food preferences is pure noise — removing it reduces model variance without losing any predictive signal.

3.2 Redundant Features

Two features are redundant if one can be expressed as a linear combination of the other(s). A simple case: if Feature 1 $= \{1, 2, 3\}$ and Feature 2 $= \{2, 4, 6\}$, then Feature 2 $= 2 \times$ Feature 1 and is fully redundant. In practice the relationship is rarely obvious; hundreds of columns may hide a complex linear dependency invisible to the eye.

3.3 Feature Selection Strategies

1. **Manual selection.** Pick a subset of the original features based on domain reasoning.
2. **Derived feature selection.** Generate new features from existing ones, then select among them.
3. **Systematic linear combination (PCA).** Automatically construct new features as weighted combinations of all originals, then keep only the most informative ones. Covered in detail next.

4 Principal Component Analysis (PCA)

4.1 High-Level Goal

PCA is an *unsupervised* data-reduction technique that finds linear combinations of input features (called **principal components**, PCs) that capture the most variance. It does *not* use the response variable; it looks only at the structure of the feature matrix itself.

4.2 Formal Definition

Given a k-dimensional random variable $\mathbf{r} = (r_1, \ldots, r_k)^\top$, the i-th principal component is the linear combination

$$y_i = \mathbf{w}_i^\top \mathbf{r} = \sum_{j=1}^{k} w_{ij}\, r_j$$

chosen to maximize $\mathrm{Var}(y_i)$ subject to:

1. $\mathbf{w}_i^\top \mathbf{w}_i = 1$ (unit-norm weight vector), and
2. $\mathrm{Cov}(y_i, y_j) = 0$ for all $j < i$ (uncorrelated with all earlier PCs).

Using properties of linear combinations of random variables:

$$\mathrm{Var}(y_i) = \mathbf{w}_i^\top \boldsymbol{\Sigma}_r\, \mathbf{w}_i, \qquad \mathrm{Cov}(y_i, y_j) = \mathbf{w}_i^\top \boldsymbol{\Sigma}_r\, \mathbf{w}_j$$

where $\boldsymbol{\Sigma}_r$ is the covariance matrix of $\mathbf{r}$.

4.3 Eigenvalue Decomposition

Because $\boldsymbol{\Sigma}_r$ is positive definite, it admits an eigenvalue decomposition. Let $(\lambda_1, \mathbf{e}_1), \ldots, (\lambda_k, \mathbf{e}_k)$ be the eigenvalue–eigenvector pairs, ordered so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_k > 0$. Then:

$$y_i = \mathbf{e}_i^\top \mathbf{r} = \sum_{j=1}^{k} e_{ij}\, r_j, \qquad i = 1, \ldots, k$$

$$\mathrm{Var}(y_i) = \mathbf{e}_i^\top \boldsymbol{\Sigma}_r\, \mathbf{e}_i = \lambda_i, \qquad \mathrm{Cov}(y_i, y_j) = \mathbf{e}_i^\top \boldsymbol{\Sigma}_r\, \mathbf{e}_j = 0 \;\; (i \neq j)$$

The proportion of total variance explained by the i-th PC is

$$\frac{\mathrm{Var}(y_i)}{\sum_{i=1}^{k} \mathrm{Var}(r_i)} = \frac{\lambda_i}{\lambda_1 + \cdots + \lambda_k}$$

4.4 Geometric Interpretation

Starting from a scatter plot of two correlated measurements X and Y, PCA finds a rotation of the coordinate axes such that:

- The new x-axis (PC1, the "red direction" in the slides) points in the direction of greatest spread.
- The new y-axis (PC2, the "green direction") is orthogonal to PC1 and captures remaining variance.

After the rotation, PC2 values cluster near zero, revealing that the two original measurements carry essentially one dimension of information. Dimensionality reduction then proceeds by simply dropping the low-variance axes (projecting into a smaller space) while retaining all original variables in the weight vectors.

4.5 Must Center and Scale Before PCA

Because PCA maximizes variance, it is naturally drawn to predictors measured in large units. If one predictor is income (in dollars, ranging over millions) and another is height (in feet, ranging over a few units), the early PCs will be dominated by income — not because it is more informative, but because its raw values are larger. **Centering and scaling every predictor to zero mean and unit variance before running PCA** eliminates this artifact and lets PCA find the truly informative directions.

4.6 Caveats: When PCA Can Mislead

- **PCA is blind to the response.** It maximizes predictor-set variation with no knowledge of the modeling objective. If the predictive signal is concentrated in directions of *low* variance, the PCs that explain the most variance will be useless for prediction. In that case, a supervised alternative such as **Partial Least Squares (PLS)** — which simultaneously maximizes predictor variance *and* correlation with the response — should be preferred.
- **PCA can summarize irrelevant structure.** Without domain guidance, the first PCs may reflect measurement scale artifacts or distributional oddities rather than the underlying relationships in the data.

4.7 How Many Components to Keep: The Scree Plot

A **scree plot** displays the ordered component number on the x-axis and the proportion of variance explained on the y-axis. For most datasets the curve shows a steep descent for the first few PCs, then levels off ("tapers off"). The standard heuristic is to keep all components up to the "elbow" — the point just before the curve flattens. In the example from the slides, variation tapers off at component 5, so 5 PCs are retained out of more than 100.

4.8 Worked Example: Stock Market (IBM, HP, Intel, JPM, BOA)

The slides present monthly log returns from 1990–2008 for five companies: IBM, HP, Intel, JP Morgan Chase, and Bank of America. The sample covariance matrix $\hat{\mathbf{\Sigma}}_r = \frac{1}{T-1}\sum_{t=1}^{T}(\mathbf{r}_t - \bar{\mathbf{r}})(\mathbf{r}_t - \bar{\mathbf{r}})^\top$ yields:

	PC1	PC2	PC3	PC4	PC5
Eigenvalue	284.17	112.93	57.43	46.81	29.87
Proportion	0.535	0.213	0.108	0.088	0.056
Cumulative	0.535	0.748	0.856	0.944	1.000
IBM (e)	+0.330	+0.139	−0.264	+0.895	−0.014
HP (e)	+0.483	+0.279	−0.701	−0.430	−0.116
Intel (e)	+0.581	+0.478	+0.652	−0.096	−0.016
JPM (e)	+0.448	−0.550	+0.013	−0.064	+0.702
BOA (e)	+0.347	−0.610	+0.119	−0.009	−0.702

Key results:

- **PC1** ($\lambda_1 = 284.17$, 53.5% of variance): all five weights are positive and roughly equal, so PC1 tracks the *general market momentum* — all stocks tend to move together.
- **PC2** ($\lambda_2 = 112.93$, 21.3% of variance): technology stocks (IBM, HP, Intel) have positive weights; financials (JPM, BOA) have negative weights. PC2 captures the *technology-versus-finance sector spread*.
- Together PC1 and PC2 account for $\approx 75\%$ of all variance in the five time series. The remaining 25% is idiosyncratic to individual companies.

4.9 Worked Example: Eigenfaces

The slides also illustrate PCA on 25 randomly chosen 64×64 pixel face images. Instead of storing all 25, PCA computes:

- **Mean face:** the simple average of all 25 images.
- **Principal basis 1 (PC1):** the weighted combination of the 25 images that has the highest pixel variance — the most "typical" synthetic face.
- **Principal basis 2 (PC2):** the next weighted combination, orthogonal to PC1, capturing the next-largest source of variation.
- **Principal basis 3 (PC3):** and so on.

Any individual face in the dataset can be approximately reconstructed from a small number of these basis images, achieving compression while retaining perceptual quality.

4.10 Practical Takeaway for the Project

Students can apply PCA to the feature matrix assembled for the midterm (urban score, artist score, statistical general scores, etc.) to:

- Identify and discard near-zero-variance components (use the scree plot).
- Replace correlated original features with a smaller set of uncorrelated PCs.
- Feed PC scores into any classifier as an alternative feature representation, generating additional diverse Kaggle submissions for the ensemble.
- **Remember to center and scale first.**

5 Supervised Learning with PySpark and MLlib (Homework 9)

5.1 The Shift from Unsupervised to Supervised

All prior approaches in the course were *unsupervised*: heuristic rules and matrix factorization never used labeled ground truth. Homework 9 introduces *supervised learning*: a file posted on Canvas (`test2New.txt`) provides binary labels (1 = recommend, 0 = do not recommend) for a subset of the user–track pairs. With labels available, any standard classification algorithm can be trained.

5.2 Why PySpark Classifiers?

The feature matrix for the full Kaggle project has hundreds of thousands to millions of rows. Conventional `scikit-learn` classifiers load all data into a single machine's RAM and fail at this scale. PySpark's `pyspark.ml` library mirrors the scikit-learn API but executes across the Spark cluster, handling datasets that are hundreds of gigabytes in size.

5.3 Worked Example: Bank Marketing Dataset

The slides demonstrate the full pipeline on a real dataset (`bank.csv`) from a Portuguese bank's direct-marketing campaign. The classification goal is to predict whether a client will subscribe to a term deposit (Yes/No). The dataset has 11 162 rows.

Input variables: age, job, marital, education, default, balance, housing, loan, contact, day, month, duration, campaign, pdays, previous, poutcome.
Output variable: deposit (yes/no). Class balance: 5873 "no", 5289 "yes" — approximately balanced.

5.4 Full Data Preparation Pipeline

Step 1 — Start a SparkSession and load data:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('ml-bank').getOrCreate()
df = spark.read.csv('bank.csv', header=True, inferSchema=True)
df.printSchema()
```

Step 2 — Explore:

```
import pandas as pd
pd.DataFrame(df.take(5), columns=df.columns).transpose()
df.groupBy('deposit').count().show()
numeric_features = [t[0] for t in df.dtypes if t[1] == 'int']
df.select(numeric_features).describe().toPandas().transpose()
```

Check class balance and summary statistics. Scatter-matrix plots of numeric features show whether any are highly correlated; in the bank dataset none are, so all are kept.

Step 3 — Remove uninformative columns:

```
df = df.select('age', 'job', 'marital', 'education', 'default',
               'balance', 'housing', 'loan', 'contact', 'duration',
               'campaign', 'pdays', 'previous', 'poutcome', 'deposit')
```

The `day` and `month` columns are dropped as not useful.

Step 4 — Encode categorical features and assemble:

```
from pyspark.ml.feature import OneHotEncoderEstimator, \
                               StringIndexer, VectorAssembler
categoricalColumns = ['job', 'marital', 'education', 'default',
                      'housing', 'loan', 'contact', 'poutcome']
stages = []
for categoricalCol in categoricalColumns:
    stringIndexer = StringIndexer(inputCol=categoricalCol,
                                  outputCol=categoricalCol + 'Index')
    encoder = OneHotEncoderEstimator(
        inputCols=[stringIndexer.getOutputCol()],
        outputCols=[categoricalCol + 'classVec'])
    stages += [stringIndexer, encoder]

label_stringIdx = StringIndexer(inputCol='deposit', outputCol='label')
stages += [label_stringIdx]

numericCols = ['age', 'balance', 'duration', 'campaign',
               'pdays', 'previous']
assemblerInputs = [c + 'classVec' for c in categoricalColumns] \
                  + numericCols
assembler = VectorAssembler(inputCols=assemblerInputs,
                            outputCol='features')
stages += [assembler]
```

Step 5 — Build Pipeline and split:

```
from pyspark.ml import Pipeline
pipeline = Pipeline(stages=stages)
pipelineModel = pipeline.fit(df)
df = pipelineModel.transform(df)
selectedCols = ['label', 'features'] + cols
df = df.select(selectedCols)

train, test = df.randomSplit([0.7, 0.3], seed=2018)
# Training Dataset Count: 7764   Test Dataset Count: 3398
```

After the pipeline runs, the DataFrame contains a `label` column (double) and a `features` column (sparse vector of all one-hot-encoded categoricals plus the numeric columns).

5.5 Four Classifiers: Code and Results

1. Logistic Regression

```
from pyspark.ml.classification import LogisticRegression
lr = LogisticRegression(featuresCol='features', labelCol='label',
                        maxIter=10)
lrModel = lr.fit(train)
```

```
predictions = lrModel.transform(test)

from pyspark.ml.evaluation import BinaryClassificationEvaluator
evaluator = BinaryClassificationEvaluator()
print('Test Area Under ROC', evaluator.evaluate(predictions))
# Test Area Under ROC: 0.8858
```

Beta coefficients can be inspected with `np.sort(lrModel.coefficients)` and plotted to see which features receive the largest weights.

2. Decision Tree Classifier

```
from pyspark.ml.classification import DecisionTreeClassifier
dt = DecisionTreeClassifier(featuresCol='features', labelCol='label',
                            maxDepth=3)
dtModel = dt.fit(train)
predictions = dtModel.transform(test)
evaluator.evaluate(predictions,
    {evaluator.metricName: 'areaUnderROC'})
# Test Area Under ROC: 0.78072
```

The single decision tree with `maxDepth=3` performs poorly because it is too weak given the range of features. The slide notes that its accuracy can be improved with ensemble methods (Random Forest, GBT).

3. Random Forest Classifier

```
from pyspark.ml.classification import RandomForestClassifier
rf = RandomForestClassifier(featuresCol='features', labelCol='label')
rfModel = rf.fit(train)
predictions = rfModel.transform(test)
evaluator.evaluate(predictions,
    {evaluator.metricName: 'areaUnderROC'})
# Test Area Under ROC: 0.8846
```

Random Forest aggregates many decision trees, reducing variance and recovering most of the accuracy lost by a single tree.

4. Gradient-Boosted Tree (GBT) Classifier

```
from pyspark.ml.classification import GBTClassifier
gbt = GBTClassifier(maxIter=10)
gbtModel = gbt.fit(train)
predictions = gbtModel.transform(test)
evaluator.evaluate(predictions,
    {evaluator.metricName: 'areaUnderROC'})
# Test Area Under ROC: 0.89407
```

GBT achieves the highest AUC of the four classifiers on this dataset.

Classifier	Test AUC (bank dataset)
Logistic Regression	0.8858
Decision Tree	0.7807
Random Forest	0.8846
Gradient-Boosted Tree	0.8941

5.6 Hyperparameter Tuning with Cross-Validation

Because GBT achieved the best results, the slides show how to tune it with a **grid search** over hyperparameters using `ParamGridBuilder` and `CrossValidator`:

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
paramGrid = (ParamGridBuilder()
             .addGrid(gbt.maxDepth, [2, 4, 6])
             .addGrid(gbt.maxBins, [20, 60])
             .addGrid(gbt.maxIter, [10, 20])
             .build())

cv = CrossValidator(estimator=gbt,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=5)
# Trains over 3 x 2 x 2 = 12 param combinations, each 5-fold
# Warning: can take 6-10 minutes
cvModel = cv.fit(train)
predictions = cvModel.transform(test)
evaluator.evaluate(predictions)
```

Key GBT hyperparameters exposed by `gbt.explainParams()`:

`maxDepth` Maximum depth of each tree (≥ 0; depth 0 = single leaf). Default: 5.

`maxBins` Number of bins for discretizing continuous features; must be ≥ 2 and $\geq$ the number of categories for any categorical feature. Default: 32.

`maxIter` Maximum number of boosting iterations (trees to build). Default: 20; the slide code uses 10.

`stepSize` Learning rate (shrinkage factor applied to each new tree). In $[0, 1]$. Default: 0.1.

`subsamplingRate` Fraction of training data used for each tree. In $(0, 1]$. Default: 1.0.

`minInstancesPerNode` Minimum number of instances each child node must have after a split. Default: 1.

`lossType` Loss function (logistic for binary classification). Default: logistic.

5.7 Applying to the Kaggle Project

1. **Training data:** Use the ground-truth file (`test2New.txt`) posted on Canvas. It contains binary labels for a *subset* of user–track pairs; the rest must be predicted by the trained model.

2. **Feature matrix:** Reuse the measurements from the midterm (urban score, artist score, statistical general scores, etc.).
3. **Adapt the bank pipeline:** replace the bank CSV load and column list with your Kaggle data; categorical encoding and `VectorAssembler` steps remain the same pattern.
4. **Train all four classifiers** on the labeled subset, apply to the full test set, and submit each to Kaggle.
5. **Submit to Canvas:** Python code + four Kaggle score screenshots.

6 Ensemble Preview

The final lecture (Week 13) will show how to blend all historical Kaggle predictions into a single submission that outperforms any individual one. The core idea:

$$\hat{y}_{\text{blend}} = \sum_{k=1}^{K} w_k \, \hat{y}^{(k)}$$

where $\hat{y}^{(k)}$ is the k-th historical submission and w_k are weights optimized to minimize prediction error. Because each submission comes from a different method (heuristic, matrix factorization, PySpark classifiers, PCA-transformed features, etc.), their errors are weakly correlated. Blending weakly correlated predictors reduces variance and lifts the score above any single submission.

Key advice: students who have accumulated many diverse submissions will benefit more from the ensemble step than those with only a few.

7 Key Reference

Center-scaling. Subtract mean, divide by std. dev. — required before PCA to avoid scale-dominance.

Spatial sign. $x^*_{ij} = x_{ij} / \sum_j x^2_{ij}$ — projects each sample onto a unit sphere, neutralizing outliers; must center-scale first; acts on the whole row, not individual features.

Scree plot. Ordered bar/line chart of eigenvalues vs. component number; keep components up to the "elbow" where variance tapers off.

Noise feature. A column with no predictive signal; removing it reduces model variance.

Redundant feature. A column expressible as a linear combination of others; PCA detects these automatically.

PCA. Unsupervised; finds orthogonal weight vectors $\mathbf{e}_i$ (eigenvectors of $\mathbf{\Sigma}$) that maximize $\mathrm{Var}(y_i) = \lambda_i$.

PLS. Supervised alternative to PCA; maximizes predictor variance *and* correlation with the response simultaneously.

Covariance matrix $\mathbf{\Sigma}$. Σ_{ij} = covariance between predictors i and j; eigenvectors are the PC directions, eigenvalues are the PC variances.

Proportion of variance. $\lambda_i/(\lambda_1 + \cdots + \lambda_k)$ — fraction of total variance explained by PC_i.

Supervised learning. Uses labeled ground truth to train a classifier; contrast with heuristic and matrix-factorization approaches.

Pipeline (`pyspark.ml`). Chains `StringIndexer` → `OneHotEncoderEstimator` → `VectorAssembler` → classifier into a single `fit`/`transform` workflow.

BinaryClassificationEvaluator. Computes ROC-AUC on the predictions DataFrame; used for all four classifiers.

Logistic Regression AUC. 0.8858 on bank dataset.

Decision Tree AUC. 0.7807; weak alone but improved by ensemble methods.

Random Forest AUC. 0.8846; aggregates many trees to reduce variance.

GBT AUC. 0.8941; best single classifier; tunable via `ParamGridBuilder` + `CrossValidator`.

CrossValidator. 5-fold CV over a `ParamGridBuilder` grid (maxDepth, maxBins, maxIter); takes ∼6–10 minutes for GBT.

Ensemble / blending. Weighted combination $\hat{y}_{\text{blend}} = \sum_k w_k \hat{y}^{(k)}$; more diverse submissions ⇒ higher ceiling.

Homework 9. Four PySpark classifiers on the Kaggle music data; submit four Kaggle predictions + code to Canvas.